

The build order

One recommended sequence from where the code is today to a working agent. Ten steps, each independently useful, each ending in something demonstrable. Written 2026-08-18. Companion to [AGENT_ARCHITECTURE.md](#) , which argues *why* this shape; this says *in what order*.

The one decision this rests on

Recommendation: the first agent job is "draft content from my own posts", not "answer a buyer in WhatsApp".

Content agent	WhatsApp buyer agent
Reads data we already own	Needs Meta business verification
No external approval	Blocked on someone else's clock
Read-heavy → cheap to run	Write-heavy → every reply is a risk
Wrong answer = a bad caption	Wrong answer = a lost sale, in public
The signup page already promises it	Nothing has promised it yet

Four reasons, and the first is decisive: **it is not blocked on anyone else**. Meta verification is outside our control, and a plan whose first milestone depends on someone else's queue is not a plan.

The fourth reason matters most for an agent's first outing: a bad draft is deleted by the seller in two seconds. A bad reply to a buyer happened in public and cost a sale.

Soko AI still gets built. It goes second, on rails this proves.

The sequence

TODAY — Phase 0 steps 1-3 done: session layer, auth pages, dashboard shell
220 tests green · design system matched to the brand



1	Connect an account	claim → code → verified	●●
2	Post + snapshots	★ the corpus	●
3	Job + worker	★ the async spine	●●
4	Analytics sync as a job	first real job	●
5	Analytics page	the loop closes	●●
— PHASE 0 COMPLETE — a stranger can sign up and learn something true about their own content. Ship it.			
6	tool_run ledger + budget	the money rail	●
7	ToolSpec + decision layer	the four gates	●
8	Read tools	agent's senses	●
9	Agent runtime	the loop	●●
10	Approval queue	the human gate	●●
— AGENT COMPLETE — drafts hooks and captions from real performance, and nothing publishes without a person.			

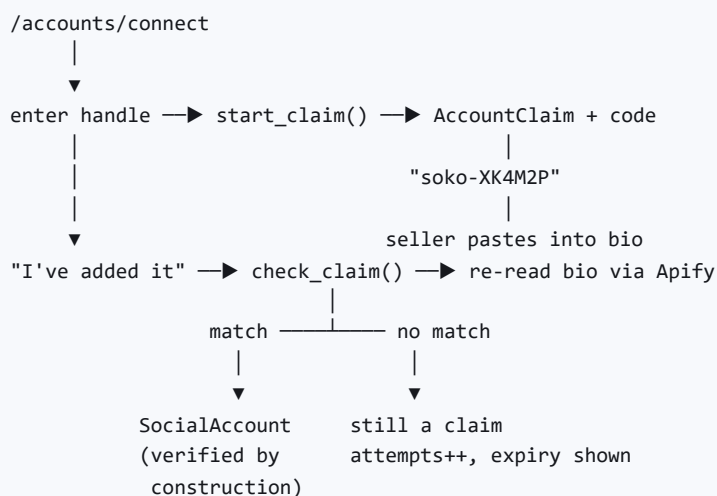
★ = expensive to retrofit. Do not reorder these two.

● = roughly one working session

Steps 1–5 are Phase 0 as already planned, with one change: the job queue moves *earlier*, to step 3, so analytics sync is written asynchronous from the start rather than converted later.

Step 1 — Connect an account

Wires `services/verification.py` — built and tested, 35 tests — to real screens.

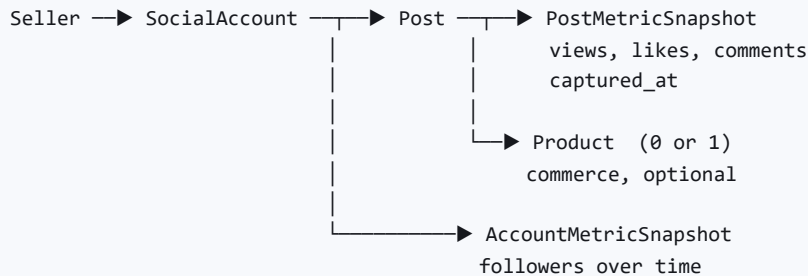


Decide here: the code prefix is still `soko-`. Changing it to `bm-` is one line and invalidates any code already in a bio. Nobody has one yet — this is the last free moment.

Demonstrable: connect a real TikTok account end to end, in a browser.

Step 2 — Post + snapshots ★

The migration. No UI. **The most time-critical step in this document.**



Two problems this fixes, and only one is recoverable:

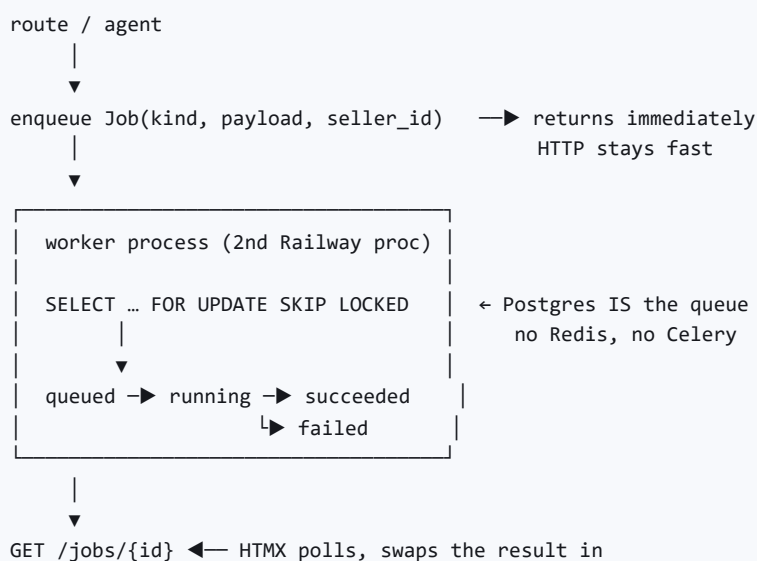
- **ingestion.py throws away non-product posts.** Right for a catalogue, wrong for analytics — a talking-head video still has views the creator is paying to understand. A filter to relax.
- **Product.views is overwritten every sync.** Nothing can answer "*am I growing?*" **History cannot be backfilled.** Every day without snapshots is a day of data gone permanently.

Start writing snapshots the moment this ships. This is the only step that costs more to delay than to do.

Demonstrable: migration applies, tests green, a sync writes rows to all three tables.

Step 3 — Job + worker ★

Generalise `ScrapeJob` into a `Job` any slow work can use.

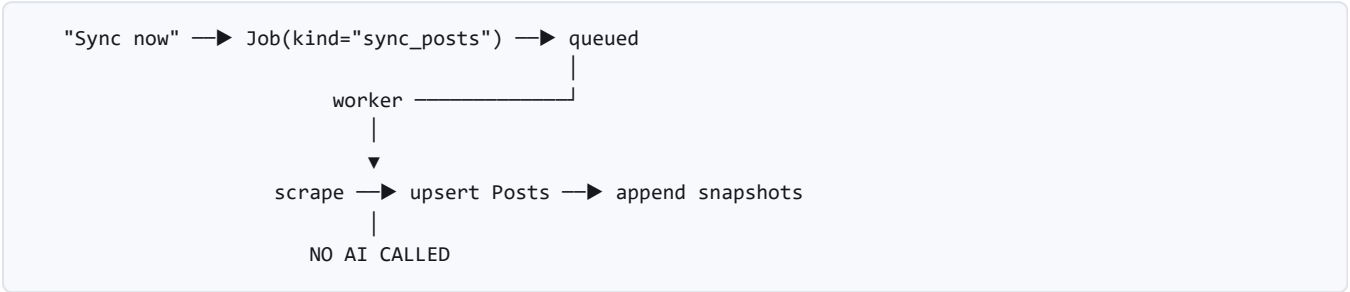


Why before the sync that needs it: retrofitting async onto synchronous code touches every caller. Written this way from the start, it costs nothing.

Why no broker: hundreds of jobs a day. Postgres handles that correctly, and it is a datastore we already operate, back up and monitor. Redis would be a second thing to be down.

Demonstrable: enqueue a slow job from a route; the page returns instantly and fills in when the worker finishes.

Step 4 — Analytics sync, as a job



Analytics ingestion does not call Gemini at all. Scrape, store, done. Only *commerce* ingestion pays for drafting. That matters at \$10/month: **the feature everyone uses daily is the cheap one.**

Keeps the existing once-per-day cooldown — it is the same scrape either way. Stores *every* post, product or not.

Demonstrable: press sync, watch rows appear in all three tables.

Step 5 — The analytics page

Followers	Total views	Posts
22,000	1.2M	1,453

with change since the last snapshot

followers / views over time (server SVG)

post table – sortable by views, comments, engagement rate, date

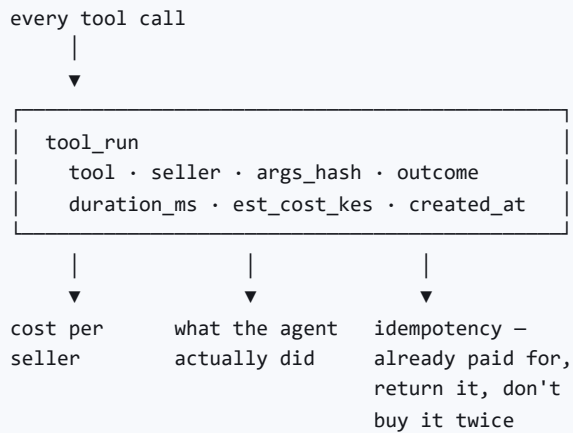
YOUR AVERAGE ← the baseline everything else is measured against

"Your average" is not a nice-to-have. It is the number the agent will need in step 8 to say "*this post did 4× your normal*", and the number competitor benchmarking needs in Phase 2. Build it now or build it twice.

Chart as server-rendered SVG — fast on a cheap phone, no dependency.

Demonstrable: Phase 0 is complete. A stranger signs up and learns something true about their own content. **Ship it and put it in front of a real creator.**

Step 6 — The money rail



Then the budget, checked **before** the call:

```
spent_today(seller) + est_cost > daily_budget → refuse
```

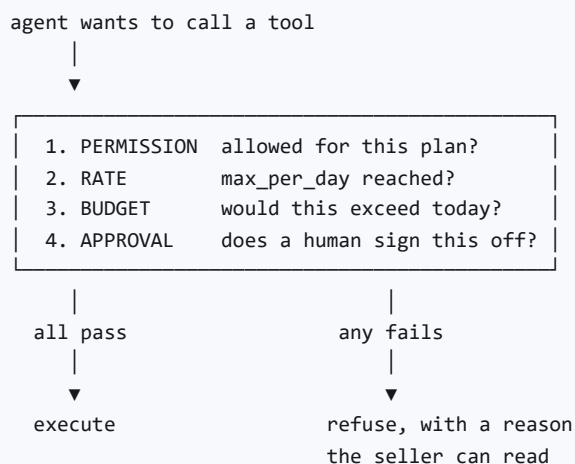
`ingestion.py` already contains this instinct hand-rolled: the cooldown, checked before the paid call, with a test asserting that ordering. This makes it infrastructure.

Do this before an agent exists. Un-ledgered spend cannot be reconstructed afterwards, and Apify and Gemini are costing money *today*.

Demonstrable: a report of what each seller cost us this week.

Step 7 — The decision layer

Plain deterministic code. **No model decides whether the model may act.**



Every tool declares itself:

```
class ToolSpec:
    name: str
    reads: bool      # only looks
    spends: bool     # costs money
    writes: bool     # changes our data
    publishes: bool  # visible OUTSIDE Biashara Mall
    max_per_day: int
```

`publishes: True` **always stops at gate 4.** That is not a policy to relax for convenience later — it is the promise the signup page prints:

User approval first. Publish to TikTok, Instagram or YouTube when ready.

A promise printed on the signup page has to be enforced in code. This is where.

Read-only tools pass every gate trivially — which is what keeps an agent loop over our own data both safe and fast.

Demonstrable: a tool refuses itself when over budget, and says why.

Step 8 — Read tools

The agent's senses. All cheap, all over data we already own.

```
get_my_stats(account)      followers, views, posts, deltas
get_top_posts(account, n)  ranked by views or engagement
get_my_average(account)    the baseline from step 5
get_post(post_id)         caption, hashtags, metrics
get_recent_posts(account, n) the raw material for drafting
```

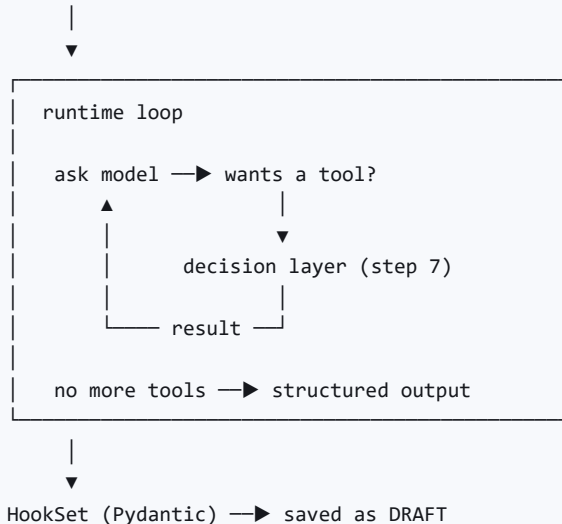
Every one is `WHERE ... ORDER BY ...` over our own tables. **No embeddings, no vector store** — see `AGENT_ARCHITECTURE.md` §2.

Typed in and out with Pydantic, exactly as `ProductDraft` already is. Each calls `services/`, the same functions the routes call.

Demonstrable: call each tool from a script and get typed results back.

Step 9 — The agent runtime

"Draft me three hooks from what's working"



Runs **inside a Job** (step 3), because it is slow. The seller presses a button, gets a "working on it" state, and HTMX fills it in.

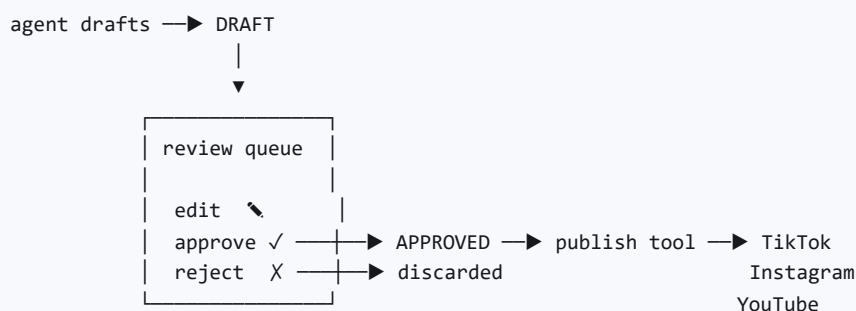
Two hard rules, both already in `CLAUDE.md` :

- **Structured output only.** The model returns a Pydantic model or it failed. Never free text we then parse.
- **One provider seam.** `agent/draft.py` is already the only file that knows we use Gemini. A second agent must not create a second such file.

Demonstrable: ask for hooks, get three grounded in your actual top posts.

Step 10 — The approval queue

Where `publishes: True` waits for a person.



The same shape the catalogue already uses — publish is a human gate, and it still requires a price. This reuses that pattern rather than inventing a second one.

Demonstrable: the full loop. Sync → analyse → draft → review → publish, with a human in the one place that matters.

What is deliberately not here

	WHY
RAG / pgvector / Chroma	Nothing to retrieve semantically. AGENT_ARCHITECTURE.md §2
Redis / Celery	Postgres is the queue at our volume
Next.js	The frontend is built, and nothing here needs a SPA
Competitor search	Phase 2. Needs the seller's own baseline first — step 5
Canva / CapCut tools	Phase 3.5. They are publishes -adjacent and need step 10 first
WhatsApp / Soko AI	Gated on Meta. Goes second, on these rails
Conversational memory	Only Soko AI needs it. Steps 1–10 need rows, not memory

Where to stop, if you have to

Each step leaves the product better than it was. That is the test of whether a sequence is real or is scaffolding.

- **Stop after 5** — a working analytics product a creator would pay for.
- **Stop after 7** — that product, with spending under control.
- **Stop after 10** — an agent that drafts from real performance and cannot publish behind your back.